



Boost Your Amazon EKS Cluster Reliability with Node Auto Repair



By

Mahendran Selvakumar

<https://devopstronaut.com/>

<https://www.linkedin.com/in/mahendran-selvakumar/>



What is Node Auto Repair?

Node Auto Repair is a feature in Amazon EKS (Elastic Kubernetes Service) that helps maintain the health of your cluster by automatically identifying and replacing unhealthy nodes. When a node becomes unresponsive or fails health checks, Node Auto Repair terminates the faulty node and launches a new one to restore the cluster's capacity and functionality. This feature reduces manual intervention and ensures high availability and reliability in your Kubernetes workloads.

Benefits

- **Reduced Downtime:** Automatically replaces failed nodes without requiring manual intervention.
- **Improved Reliability:** Ensures application workloads have sufficient healthy nodes to run on.
- **Operational Efficiency:** Simplifies cluster maintenance by automating node recovery.

Use Case:

- Ensures high availability and reliability in Amazon EKS clusters.
- Automatically detects and replaces unhealthy nodes.
- Reduces manual intervention and operational overhead.
- Minimizes downtime in production environments.
- Maintains consistent application performance.

Step 1: Create the EKS Cluster Without Any Node Groups

Create an EKS cluster without a node group using the `eksctl` command **`eksctl create cluster --name=eks-auto-node --region=eu-west-1 --without-nodegroup`**. By default, `eksctl` creates a node group with `m5.large` instances, so we used the `--without-nodegroup` option to skip creating a default node group



```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl create cluster --name=eks-auto-node --region=eu-west-1 --without-nodegroup
2024-12-24 17:12:20 [i] eksctl version 0.196.0
2024-12-24 17:12:20 [i] using region eu-west-1
2024-12-24 17:12:20 [i] setting availability zones to [eu-west-1a eu-west-1c eu-west-1b]
2024-12-24 17:12:20 [i] subnets for eu-west-1a - public:192.168.0.0/19 private:192.168.96.0/19
2024-12-24 17:12:20 [i] subnets for eu-west-1c - public:192.168.32.0/19 private:192.168.128.0/19
2024-12-24 17:12:20 [i] subnets for eu-west-1b - public:192.168.64.0/19 private:192.168.160.0/19
2024-12-24 17:12:20 [i] using Kubernetes version 1.30
2024-12-24 17:12:20 [i] creating EKS cluster "eks-auto-node" in "eu-west-1" region with
2024-12-24 17:12:20 [i] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=eu-west-1 --cluster=eks-auto-node'
2024-12-24 17:12:20 [i] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "eks-auto-node" in "eu-west-1"
2024-12-24 17:12:20 [i] CloudWatch logging will not be enabled for cluster "eks-auto-node" in "eu-west-1"
2024-12-24 17:12:20 [i] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=eu-west-1 --cluster=eks-auto-node'
2024-12-24 17:12:20 [i] default add-ons vpc-cni, kube-proxy, coreDNS were not specified, will install them as EKS add-ons
2024-12-24 17:12:20 [i]
2 sequential tasks: { create cluster control plane "eks-auto-node",
  2 sequential sub-tasks: {
    1 task: { create add-ons },
    wait for control plane to become ready,
  }
}
2024-12-24 17:12:20 [i] building cluster stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:12:20 [i] deploying stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:12:50 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:13:20 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:14:21 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:15:21 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:16:21 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:17:21 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:18:21 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:19:21 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:33:11 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-cluster"
2024-12-24 17:33:14 [i] recommended policies were found for "vpc-cni" add-on, but since OIDC is disabled on the cluster, eksctl cannot configure the requested permissions; the recommended way to provide IAM permissions for "vpc-cni" add-on is via pod identity associations; after add-on creation is completed, add all recommended policies to the config file, under 'addon.PodIdentityAssociations', and run 'eksctl update add-on'
2024-12-24 17:33:14 [i] creating add-on
2024-12-24 17:33:14 [i] successfully created add-on
2024-12-24 17:33:14 [i] creating add-on
2024-12-24 17:33:15 [i] successfully created add-on
2024-12-24 17:33:15 [i] creating add-on
2024-12-24 17:33:16 [i] successfully created add-on
2024-12-24 17:35:16 [i] waiting for the control plane to become ready
2024-12-24 17:35:16 [x] saved kubeconfig as "/Users/mahendranselvakumar/.kube/config"
2024-12-24 17:35:16 [i] no tasks
2024-12-24 17:35:16 [x] all EKS cluster resources for "eks-auto-node" have been created
2024-12-24 17:35:17 [i] kubectl command should work with "/Users/mahendranselvakumar/.kube/config", try 'kubectl get nodes'
2024-12-24 17:35:17 [x] EKS cluster "eks-auto-node" in "eu-west-1" region is ready
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Go to the EKS console to verify that the cluster was successfully created using eksctl

Clusters (1) Info									
Filter clusters									
	Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider		
☐	eks-auto-node	Active	1.30 Upgrade now	Standard support until July 28, 2025	Extended	34 minutes ago	EKS		

Step 2: Create a Managed Node Group

Add a node group using the following separate eksctl command **eksctl create nodegroup --name eks-autonode-ng --cluster eks-auto-node --region eu-west-1 --nodes 2 --nodes-min 1 --nodes-max 3 --node-type t3.medium --enable-ssm**



```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl create nodegroup --name eks-autonode-ng --cluster eks-auto-node --region eu-west-1 --nodes 2 --nodes-min 1 --nodes-max 3 --node-type t3.medium --enable-ssm
2024-12-24 17:41:10 [i] will use version 1.30 for new nodegroup(s) based on control plane version
2024-12-24 17:41:10 [i] SSM is now enabled by default; 'ssh.enableSSM' is deprecated and will be removed in a future release
2024-12-24 17:41:12 [i] nodegroup "eks-autonode-ng" will use "" [AmazonLinux2/1.30]
2024-12-24 17:41:12 [i] 1 nodegroup (eks-autonode-ng) was included (based on the include/exclude rules)
2024-12-24 17:41:12 [i] will create a CloudFormation stack for each of 1 managed nodegroups in cluster "eks-auto-node"
2024-12-24 17:41:12 [i]
2 sequential tasks: { fix cluster compatibility, 1 task: { 1 task: { create managed nodegroup "eks-autonode-ng" } } }
}
2024-12-24 17:41:12 [i] checking cluster stack for missing resources
2024-12-24 17:41:12 [i] cluster stack has all required resources
2024-12-24 17:41:12 [i] building managed nodegroup stack "eksctl-eks-auto-node-nodegroup-eks-autonode-ng"
2024-12-24 17:41:13 [i] deploying stack "eksctl-eks-auto-node-nodegroup-eks-autonode-ng"
2024-12-24 17:41:13 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-nodegroup-eks-autonode-ng"
2024-12-24 17:41:43 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-nodegroup-eks-autonode-ng"
2024-12-24 17:42:27 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-nodegroup-eks-autonode-ng"
2024-12-24 17:43:59 [i] waiting for CloudFormation stack "eksctl-eks-auto-node-nodegroup-eks-autonode-ng"
2024-12-24 17:43:59 [i] no tasks
2024-12-24 17:43:59 [✓] created 0 nodegroup(s) in cluster "eks-auto-node"
2024-12-24 17:43:59 [i] nodegroup "eks-autonode-ng" has 2 node(s)
2024-12-24 17:43:59 [i] node "ip-192-168-21-134.eu-west-1.compute.internal" is ready
2024-12-24 17:43:59 [i] node "ip-192-168-74-190.eu-west-1.compute.internal" is ready
2024-12-24 17:43:59 [i] waiting for at least 1 node(s) to become ready in "eks-autonode-ng"
2024-12-24 17:43:59 [i] nodegroup "eks-autonode-ng" has 2 node(s)
2024-12-24 17:43:59 [i] node "ip-192-168-21-134.eu-west-1.compute.internal" is ready
2024-12-24 17:43:59 [i] node "ip-192-168-74-190.eu-west-1.compute.internal" is ready
2024-12-24 17:43:59 [✓] created 1 managed nodegroup(s) in cluster "eks-auto-node"
2024-12-24 17:43:59 [i] checking security group configuration for all nodegroups
2024-12-24 17:43:59 [i] all nodegroups have up-to-date cloudformation templates
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Explanation of the flags:

- **--cluster:** Specifies the name of the existing EKS cluster to which the node group will be added.
- **--name:** Names the node group for easy identification.
- **--region:** Specifies the AWS region.
- **--nodes:** Sets the initial desired number of nodes (in this case, 2).
- **--nodes-min** and **--nodes-max:** Define the minimum and maximum number of nodes for auto-scaling.
- **--node-type:** The EC2 instance type for the nodes (e.g., m5.large)

If you check in the console, you'll see that the node group has been created

The screenshot shows the AWS Management Console for the 'eks-autonode-ng' node group. The console displays the following information:

- Node group configuration:**
 - Kubernetes version: 1.30
 - AMI type: AL2_x86_64
 - Launch template: eksctl-eks-auto-node-nodegroup-eks-autonode-ng
 - Status: Active
 - AMI release version: 1.30.7-20241213
 - Instance types: t3.medium
 - Launch template version: 1
 - Disk size: Specified in launch template
- Details:**
 - Node group ARN: arn:aws:eks:eu-west-1:038462791702:nodegroup/eks-auto-node/eks-autonode-ng/1ec9fce1-401d-0b1e-c059-cb0bdfc86f92
 - Autoscaling group name: eks-eks-autonode-ng-1ec9fce1-401d-0b1e-c059-cb0bdfc86f92
 - Node IAM role ARN: arn:aws:iam::038462791702:role/eksctl-eks-auto-node-nodegroup-eks-NodeInstanceRole-H3wmJujl0xL
 - Capacity type: On-Demand
 - Desired size: 2 nodes
 - Minimum size: 1 node
 - Maximum size: 3 nodes
 - Subnets: subnet-083e37b12f4ef121e, subnet-0b738e30dc81fbfbb, subnet-0eaf0496229f4c0b0
 - Configure remote access to nodes: off
- Node auto repair configuration:**
 - Node auto repair: Disabled



In the instances section of the EC2 console, you can view the Instances created for the node group

Instances (2) Info									
Find Instance by attribute or tag (case-sensitive) Running									
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Put
☐	eks-auto-node-eks-autonode-ng-Node	i-0cb40049dd9848565	Running	t3.medium	3/3 checks passed	View alarms	eu-west-1a	ec2-108-129-129-192.eu-west-1...	100
☐	eks-auto-node-eks-autonode-ng-Node	i-043c214536cbdeb3b	Running	t3.medium	3/3 checks passed	View alarms	eu-west-1b	ec2-3-249-213-107.eu-west-1...	3.2

If the Node Auto Repair configuration is disabled on the Node Group page, click **Edit** to enable it

eks-autonode-ng Edit Delete

Your current IAM principal doesn't have access to Kubernetes objects on this cluster.
This may be due to the current user or role not having Kubernetes RBAC permissions to describe cluster resources or not having an entry in the cluster's auth config map. [Learn more](#)

Node group configuration [Info](#)

Kubernetes version
1.30

AMI type [Info](#)
AL2_x86_64

Launch template
[eksctl-eks-auto-node-nodegroup-eks-autonode-ng](#)

Status
Active

AMI release version [Info](#)
1.30.7-20241213

Instance types
t3.medium

Launch template version
1

Disk size
Specified in launch template

Details

Node group ARN
[arn:aws:eks:eu-west-1:038462791702:nodegroup/eks-auto-node-eks-autonode-ng/1ec9fce1-401d-0b1e-c059-c8dbdfc86f92](#)

Autoscaling group name
[eks-eks-autonode-ng-1ec9fce1-401d-0b1e-c059-c8dbdfc86f92](#)

Capacity type
On-Demand

Subnets
[subnet-083a37b12f4ef121e](#)
[subnet-0b738e30dc81fbfbf](#)
[subnet-0ea0496229f4c0b0](#)

Created
7 minutes ago

Node IAM role ARN
[arn:aws:iam::038462791702:role/eksctl-eks-auto-node-nodegroup-eks-NodeInstanceRole-H3wmJjuljoxL](#)
[View in IAM](#)

Desired size
2 nodes

Minimum size
1 node

Maximum size
3 nodes

Configure remote access to nodes
off

Node auto repair configuration [Info](#)

Node auto repair
Disabled

Click **Enable Node Auto Repair**, then click **Save Changes**

Node group update configuration [Info](#)

Maximum unavailable
Set the maximum number or percentage of unavailable nodes to be tolerated during the node group version update.

☒ Number
Enter a number

☐ Percentage
Specify a percentage

Value

node

Node count must be greater than 0.

Node auto repair configuration [Info](#)

When node auto repair is enabled, Amazon EKS continuously monitors the health of the nodes within a managed node group. This feature automatically detects and replaces nodes when issues occur.

☒ Enable node auto repair

Cancel Save changes

It will take some time to complete the update



Node group configuration update in progress
eks-autonode-ng configuration is now being updated. This process may take several minutes.

eks-autonode-ng

Your current IAM principal doesn't have access to Kubernetes objects on this cluster.
This may be due to the current user or role not having Kubernetes RBAC permissions to describe cluster resources or not having an entry in the cluster's auth config map. [Learn more](#)

Node group configuration

Kubernetes version 1.30	AMI type AL2_x86_64	Launch template eksctl-eks-auto-node-group-eks-autonode-ng	Status Updating
AMI release version 1.30.7-20241213	Instance types t3.medium	Launch template version 1	Disk size Specified in launch template

Details Nodes Health issues Kubernetes labels Update config Kubernetes taints Update history Tags

Update history (1)

Update ID	Submission time	Type	Status
41dd55b1-ae01-3d8e-a0c3-75f19a440438	a few seconds ago	ConfigUpdate	In progress

Node Auto Repair has been successfully enabled

eks-autonode-ng

Your current IAM principal doesn't have access to Kubernetes objects on this cluster.
This may be due to the current user or role not having Kubernetes RBAC permissions to describe cluster resources or not having an entry in the cluster's auth config map. [Learn more](#)

Node group configuration

Kubernetes version 1.30	AMI type AL2_x86_64	Launch template eksctl-eks-auto-node-group-eks-autonode-ng	Status Active
AMI release version 1.30.7-20241213	Instance types t3.medium	Launch template version 1	Disk size Specified in launch template

Details Nodes Health issues Kubernetes labels Update config Kubernetes taints Update history Tags

Details

Node group ARN arn:aws:eks:eu-west-1:038462791702:nodegroup/eks-auto-node-eks-autonode-ng/1ec9fce1-401d-0b1e-c059-c8bdfc86f92	Autoscaling group name eks-eks-autonode-ng-1ec9fce1-401d-0b1e-c059-c8bdfc86f92	Capacity type On-Demand	Subnets subnet-083a37b12f4ef121e subnet-0b738e30dc81f1f1fb subnet-0eaf0496229f4c0b0
Created 14 minutes ago	Node IAM role ARN arn:aws:iam::038462791702:role/eksctl-eks-auto-node-nodegroup-eks-NodeInstanceRole-H3wmJjuljoxl View in IAM	Desired size 2 nodes	Configure remote access to nodes off
		Minimum size 1 node	
		Maximum size 3 nodes	

Node auto repair configuration

Node auto repair
Enabled

Run the `kubectl get nodes` command to retrieve the details of the nodes

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-21-134.eu-west-1.compute.internal Ready    <none>   12m   v1.30.7-eks-59bf375
ip-192-168-74-190.eu-west-1.compute.internal Ready    <none>   12m   v1.30.7-eks-59bf375
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Step 3: Simulate Node Failure

Stop the kubelet on one of the nodes to simulate a node failure. Navigate to the EC2 Console, select the instance, and click **Connect**

Instances (1/2)							
Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☒ eks-auto-node-eks-autonode-ng-Node	i-0cb40049dd9848565	Running	t3.medium	3/3 checks passed	View alarms +	eu-west-1a	ec2-108-129-129-192.eu-west-1...
☐ eks-auto-node-eks-autonode-ng-Node	i-043c214536cbdeb3b	Running	t3.medium	3/3 checks passed	View alarms +	eu-west-1b	ec2-3-249-213-107.eu-west-1...

<https://www.linkedin.com/in/mahendran-selvakumar/>



Navigate to **Session Manager** and click **Connect**

[EC2](#) > [Instances](#) > [i-0cb40049dd9848565](#) > Connect to instance

Connect to instance Info

Connect to your instance i-0cb40049dd9848565 (eks-auto-node-eks-autonode-ng-Node) using any of these options

EC2 Instance Connect **Session Manager** SSH client EC2 serial console

Session Manager usage:

- Connect to your instance without SSH keys, a bastion host, or opening any inbound ports.
- Sessions are secured using an AWS Key Management Service key.
- You can log session commands and details in an Amazon S3 bucket or CloudWatch Logs log group.
- Configure sessions on the Session Manager [Preferences](#) page.

[Cancel](#)

[Connect](#)

Run the `sudo su -` command to switch to the root user, and then execute **sudo systemctl stop kubelet** to stop the kubelet service

Session ID: root-659ttotfkfzsjz7sp77rcgfn6i

Instance ID: i-0cb40049dd9848565

```
sh-4.2$ sudo su -  
[root@ip-192-168-21-134 ~]# sudo systemctl stop kubelet  
[root@ip-192-168-21-134 ~]#
```

Now, run the **kubectl get nodes** command, and you will see that the status of one of the nodes has changed to **NotReady**

```
mahendranseivakumar@Mahendrants-MacBook-Pro ~ % kubectl get nodes  
NAME                                STATUS    ROLES    AGE   VERSION  
ip-192-168-21-134.eu-west-1.compute.internal NotReady <none>   20m   v1.30.7-eks-59bf375  
ip-192-168-74-190.eu-west-1.compute.internal Ready     <none>   20m   v1.30.7-eks-59bf375  
mahendranseivakumar@Mahendrants-MacBook-Pro ~ %
```

Terminate the EC2 instance by running this command **aws ec2 terminate-instances --instance-ids i-0cb40049dd9848565** command



```
mahendranselvakumar@Mahendrns-MacBook-Pro ~ % aws ec2 terminate-instances --instance-ids i-0cb40049dd9848565
{
  "TerminatingInstances": [
    {
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "InstanceId": "i-0cb40049dd9848565",
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

The instance is shutting down and will be terminated shortly

Instances (1/3) info									
Find instance by attribute or tag (case-sensitive)									
All states									
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Put
☐	Vault-Server	i-0a322d02fb7556657	Stopped	t2.small	-	View alarms +	eu-west-1a	-	-
☒	eks-auto-node-eks-autonode-ng-Node	i-0cb40049dd9848565	Shutting-down	t3.medium	-	View alarms +	eu-west-1a	ec2-108-129-129-192.eu-west-1...	10E
☐	eks-auto-node-eks-autonode-ng-Node	i-043c214536cbdeb3b	Running	t3.medium	3/3 checks passed	View alarms +	eu-west-1b	ec2-3-249-213-107.eu-west-1...	3.2

If you run the **kubectl get nodes** command, you will see that one of the nodes is in the **Ready** state

```
mahendranselvakumar@Mahendrns-MacBook-Pro ~ % kubectl get nodes
NAME                                     STATUS    ROLES    AGE   VERSION
ip-192-168-74-190.eu-west-1.compute.internal Ready    <none>   23m   v1.30.7-eks-59bf375
mahendranselvakumar@Mahendrns-MacBook-Pro ~ %
```

Step 4: Monitor Node Auto-Repair

Another instance is being initialized automatically because the Node Auto Repair configuration is enabled

Instances (2) info									
Find instance by attribute or tag (case-sensitive)									
Running									
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Put
☐	eks-auto-node-eks-autonode-ng-Node	i-068975b9d506c6301	Running	t3.medium	Initializing	View alarms +	eu-west-1a	ec2-34-247-194-243.eu-west-1...	34.
☐	eks-auto-node-eks-autonode-ng-Node	i-043c214536cbdeb3b	Running	t3.medium	3/3 checks passed	View alarms +	eu-west-1b	ec2-3-249-213-107.eu-west-1...	3.2

Confirm that the replaced node is in the **Ready** state by running the **kubectl get nodes** command

```
mahendranselvakumar@Mahendrns-MacBook-Pro ~ % kubectl get nodes
NAME                                     STATUS    ROLES    AGE   VERSION
ip-192-168-2-38.eu-west-1.compute.internal Ready    <none>   66s   v1.30.7-eks-59bf375
ip-192-168-74-190.eu-west-1.compute.internal Ready    <none>   25m   v1.30.7-eks-59bf375
mahendranselvakumar@Mahendrns-MacBook-Pro ~ %
```




The node has now been repaired automatically.

Conclusion:

Enabling Node Auto Repair in Amazon EKS helps maintain a reliable and healthy Kubernetes cluster. It automatically identifies and replaces unhealthy nodes, reducing downtime and manual effort. This feature ensures consistent application performance and high availability, making it particularly useful in production environments where reliability is essential.

Keep Learning, Keep EKS-ing!!

Feel free to reach out to me, if you have any other queries or suggestions Stay connected on LinkedIn: [Mahendran Selvakumar](#)

Stay connected on Medium: <https://devopstronaut.com/>